

Lời giải trận thứ hai HDU Multi-University 2021

Nguồn gốc: HDU Multi-University 2021 MINIEYE Cup, Contest 2 Solutions

Contest / HDU 2021 MINIEYE Cup Contest 2 solutions.pdf

1 1001 I love cube

Dễ thấy đây là bài khởi động.

Mọi tam giác đều có thể thỏa điều kiện đều có dạng

$$(0, 0, 0), \quad (1, 1, 0), \quad (0, 1, 1).$$

Ta xét riêng từng độ dài cạnh. Với cạnh dài i , đóng góp là

$$8(n - i)^3.$$

Vì vậy đáp án cuối cùng là

$$\sum_{i=1}^{n-1} 8(n - i)^3 = 8 \sum_{i=1}^{n-1} i^3 = 2(n(n - 1))^2.$$

Chú ý cần lấy n theo mod trước, hoặc dùng `int128`.

2 1002 I love tree

Vì còn có một bài cây đoạn khác, nên các thẻ lười của cây đoạn trong bài này tương đối đơn giản.

Cách 1

Xét một đường đi $\langle a, b \rangle$. Sau khi phân rã cây nặng nhẹ, đường đi này tương ứng với $O(\log n)$ đoạn liên tiếp. Do đó bài toán chuyển thành cộng một hàm bậc hai lên một đoạn.

Giả sử trên đoạn $[h, t]$ ta cộng

$$(x - h)^2.$$

Khai triển:

$$(x - h)^2 = x^2 + h^2 - 2xh.$$

Vì vậy chỉ cần tách ba loại thẻ và duy trì riêng rẽ. Độ phức tạp thời gian là

$$O(n \log^2 n).$$

Cách 2

Xét chia khối các thao tác rồi xây dựng lại. Mỗi truy vấn tương đương với truy vấn tổng của các sửa đổi trước lần xây dựng lại và các sửa đổi chưa được cập nhật.

Với phần chưa được cập nhật, mỗi lần cần xét một đỉnh có nằm trên một đường đi hay không. Điều này có thể làm trong $O(1)$ bằng RMQ và LCA.

Với các thao tác đã xây dựng lại, ta dùng thêm sai phân. Khi đó thao tác trở thành cộng một cấp số cộng trên đoạn; có thể sửa trong $O(\text{kích thước khối})$ và duy trì tổng đoạn trong $O(n)$. Lấy kích thước khối bằng căn bậc hai, tổng độ phức tạp là

$$O(n\sqrt{n}).$$

3 1003 I love playing games

Khi kiểm tra bài này có khá nhiều lời giải sai độ phức tạp $O(n)$, nên dữ liệu có nhiều đồ thị phân tầng để chặn các lời giải đó.

Trước hết chạy đường đi ngắn nhất từ đỉnh z . Nếu

$$\text{dis}[x] < \text{dis}[y],$$

người đi trước chắc thắng. Nếu

$$\text{dis}[x] > \text{dis}[y],$$

người đi sau chắc thắng. Nếu

$$\text{dis}[x] = \text{dis}[y],$$

ta dùng DP để phán định.

Đặt

$$dp[xx][yy][0/1]$$

biểu diễn trạng thái hiện tại: x ở đỉnh xx , y ở đỉnh yy , và đến lượt ai đi. Giá trị DP biểu diễn thắng chắc, hòa, hoặc thua chắc. Khi chuyển trạng thái, chỉ cần duyệt thô các cạnh trong đồ thị đường đi ngắn nhất.

Độ phức tạp thời gian là

$$O(n^2).$$

4 1004 I love counting

Bài này hơi trùng ý tưởng với bài 1006 và *idea* của bài 1010 ở trận trước, coi như một bài ôn tập của trận trước.

Có rất nhiều cách làm. Lời giải của std xây một 01-trie trên toàn bộ các số. Mỗi nút của trie ghi lại mọi điểm trong cây con của nó. Với một truy vấn, phần nó bao phủ là nhiều nhất $O(\log n)$ cây con trên trie; treo truy vấn lên các cây con này, rồi cuối cùng làm đếm điểm hai chiều trên từng cây con. Cách làm giống trận trước: biến đếm số màu trong đoạn thành điều kiện “tiền nhiệm < đầu trái truy vấn”, đồng thời đếm truy vấn nằm trong đoạn.

5 1005 I love string

Dễ thấy, nếu ta muốn mô phỏng ra dãy có thứ tự từ điển nhỏ nhất, chỉ đoạn các ký tự giống nhau ở đầu mới có hai lựa chọn. Gọi độ dài đoạn giống nhau ban đầu là x , đáp án là

$$2^{x-1}.$$

6 1006 I love sequences

Vì

$$\sum_{p=1}^n \frac{n}{p} = n \log n,$$

nên chỉ cần tính tích chập riêng cho từng p .

Khuôn mẫu kinh điển của FWT là xây các ma trận sao cho

$$(T_1 A) * (T_2 B) = T_3 C.$$

Sau khi tách theo bit cao nhất, tức

$$A = [A0, A1, A2], \quad B = [B0, B1, B2], \quad C = [C0, C1, C2],$$

ta cần

$$T_1 A(x) * T_2 B(x) = T_3 C(x).$$

Theo yêu cầu của bài này,

$$C[0] = A[0]B[0],$$

$$C[1] = (A[0] + A[1] + A[2])(B[0] + B[1] + B[2]) - C[2],$$

$$C[2] = (A[0] + A[2])(B[0] + B[2]) - C[0].$$

Cài đặt khá dễ. Độ phức tạp thời gian là

$$O(n \log^2 n).$$

7 1007 I love data structure

Đề bài đơn giản, dễ hiểu.

Hai thao tác thứ hai và thứ ba có thể biểu diễn bằng các ma trận

$$\begin{pmatrix} 3 & 3 \\ 2 & -2 \end{pmatrix} \quad \text{và} \quad \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Trên mỗi nút của cây đoạn, duy trì thẻ nhân ma trận và thẻ cộng. Khi đẩy thẻ nhân xuống, dùng thẻ đó để cập nhật thẻ cộng và tổng cần hỏi. Đồng thời còn cần duy trì

$$\sum_{i=1}^n a_i, \quad \sum_{i=1}^n b_i, \quad \sum_{i=1}^n a_i^2, \quad \sum_{i=1}^n b_i^2, \quad \sum_{i=1}^n a_i b_i,$$

và cập nhật chúng khi đẩy thẻ xuống.

Ban đầu dự định đặt giới hạn thời gian 3 giây, nhưng lỡ đặt thành 5 giây, nên cách duy trì ma trận 6×6 cũng qua được.

8 1008 I love exam

DP nhập môn.

Với mỗi môn học, dùng ba lô để tính $f(i)$: khi tốn i ngày thì tối đa nhận được bao nhiêu điểm.

Khi gộp các môn học, ghi $g(j, k)$ là số điểm lớn nhất nhận được khi tốn j ngày và rút k môn. Chuyển trạng thái cuối cùng có độ phức tạp

$$O(nt^2p).$$

9 1009 I love triples

Khử các thừa số chính phương trong a_i rồi rời rạc hóa. Khi đó bài toán là đếm số bộ (i, j, k) thỏa

$$1 \leq i \leq j \leq k \leq V, \quad V = 100000,$$

và ijk là một số chính phương.

Dễ thấy (i, j, k) chắc chắn thuộc một trong hai trường hợp sau:

1. i, j, k đều không chứa thừa số lớn hơn $\sqrt{V}$.
2. Trong i, j, k , có hai số chứa cùng một thừa số lớn hơn $\sqrt{V}$.

Với loại thứ nhất, sau khi tìm kiếm sẽ thấy số lượng bộ như vậy rất nhỏ, cứ tìm kiếm vét cạn là đủ.

Với loại thứ hai, liệt kê hai số chứa thừa số lớn hơn $\sqrt{V}$, rồi dùng hash để tìm số còn lại. Vì số lượng số chứa thừa số P với $P > \sqrt{V}$ không vượt quá $\sqrt{V}$, tổng hiệu suất liệt kê là

$$O(V\sqrt{V}).$$

10 1010 I love permutation

Dãy thu được hiển nhiên là một hoán vị độ dài $P - 1$. Cần tính số nghịch thế của hoán vị theo modulo 2, tức là tính tính chẵn lẻ của hoán vị.

Gọi hoán vị là π , phần tử thứ i của nó là $\pi(i)$, số nghịch thế là $n(\pi)$, và

$$\text{sgn}(\pi) = (-1)^{n(\pi)}.$$

Khi đó

$$\begin{aligned} \text{sgn}(\pi) &= \frac{\prod_{0 \leq j < i \leq P-1} (\pi(i) - \pi(j))}{\prod_{0 \leq j < i \leq P-1} (i - j)} \\ &= \prod_{0 \leq j < i \leq P-1} \frac{\pi(i) - \pi(j)}{i - j} \\ &\equiv \prod_{0 \leq j < i \leq P-1} \frac{ia - ja}{i - j} = a^{P(P-1)/2} \equiv a^{(P-1)/2} \pmod{P}. \end{aligned}$$

Vì vậy chỉ cần trực tiếp tính

$$a^{(P-1)/2} \pmod{P},$$

là biết $\text{sgn}(\pi)$, từ đó suy ra tính chẵn lẻ của $n(\pi)$.

11 1011 I love max and multiply

Cần tính

$$C_k = \max(A_i B_j)$$

với điều kiện

$$i \& j \geq k.$$

Có thể xét việc tính tất cả

$$D_k = \max(A_i B_j)$$

với điều kiện $i \& j = k$, rồi lấy max từ sau ra trước.

Tuy nhiên điều kiện $i \& j = k$ cũng không dễ làm. Ta đổi sang tính $D_k = \max(A_i B_j)$ với điều kiện $k \in i \& j$.

Với riêng A và B , lần lượt tính các giá trị sau trong điều kiện $k \in i, j$:

$$\max(A_i), \quad \min(A_i), \quad \max(B_j), \quad \min(B_j).$$

Sau đó nhân từng cặp và lấy giá trị lớn nhất, ta thu được D_k .